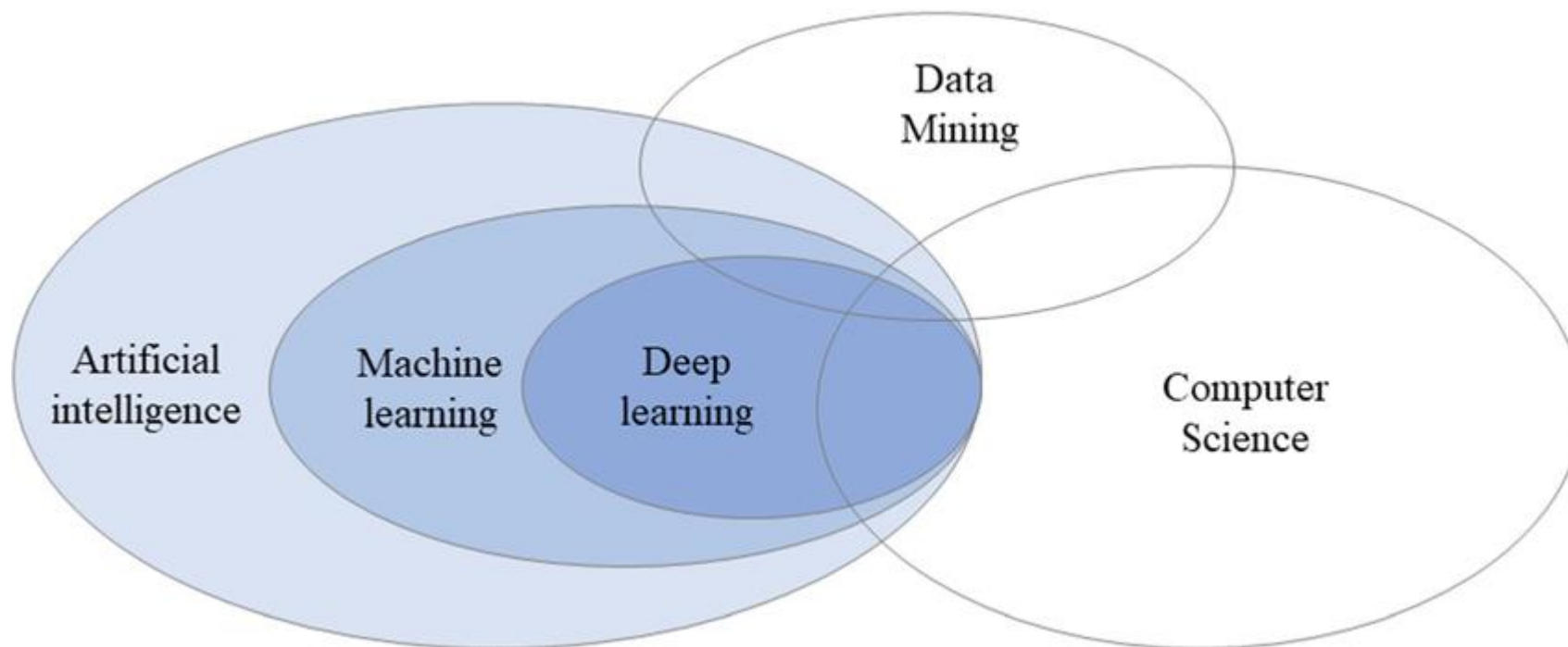


# 컴퓨터비전\_인공지능(AI) 개론

## 4. 인공지능 기초 - 신경망과 역전파

- 문제 정의하기
- 경사 하강법 이해하기
- 경사 하강법의 구현 방법
- 데이터 전처리
- 예측 계산
- 손실 계산
- 경사 계산
- 파라미터 수정
- 반복 계산
- 결과 평가
- 최적화 함수와 step 함수 이용하기

## ▶ 관계도



[https://www.researchgate.net/figure/Venn-diagram-representing-the-relationships-between-AI-ML-and-DL-Adapted-from\\_fig1\\_349365459](https://www.researchgate.net/figure/Venn-diagram-representing-the-relationships-between-AI-ML-and-DL-Adapted-from_fig1_349365459)

## ▶ 개념 이해하기

1) AI (인공지능): 사람처럼 생각하는 모든 기술 (가장 넓은 개념)

예: 체스를 두는 컴퓨터, 번역기, 챗봇

2) ML (기계학습): 데이터로부터 스스로 학습하는 AI

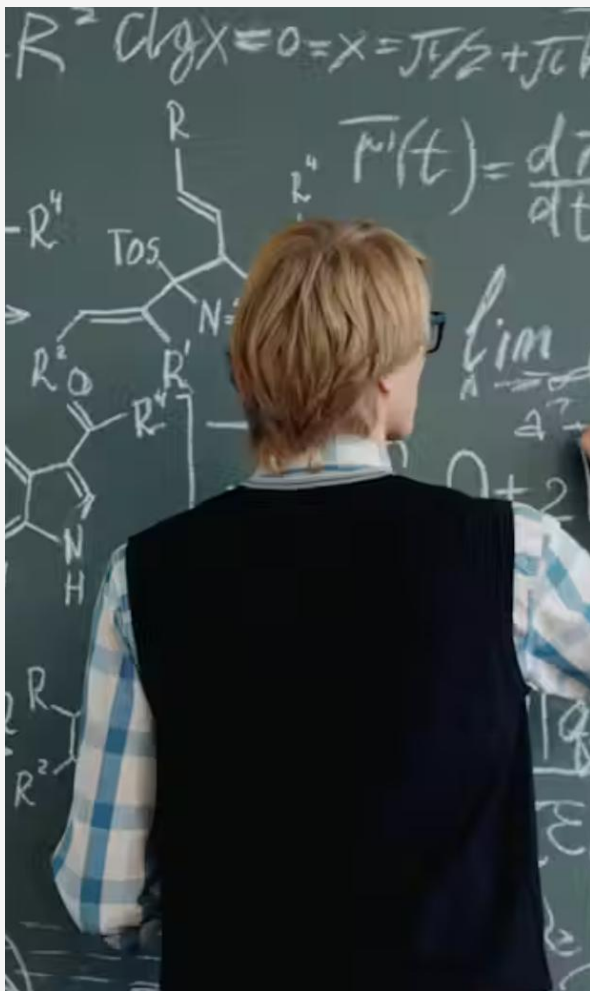
예: 스팸 메일 필터 (스팸 메일 예시를 보고 학습)

3) DL (딥러닝): 인간 뇌처럼 여러 층으로 학습하는 ML

예: 얼굴 인식, 자율주행차 (복잡한 패턴 학습)

**인공지능(AI) ⊃ 머신러닝(ML) ⊃ 딥러닝(DL)**

## ▶ AI 어떻게 학습할까요?



### 지도학습

선생님이 문제와 답을 함께 알려주며 가르치는 것처럼, 입력과 정답이 쌍으로 주어진 데이터로 학습합니다.

### 비지도학습

책장의 책들을 장르별로 자동 분류하듯, 정답 없이 데이터의 숨겨진 패턴을 스스로 발견합니다.

### 자지도학습

책의 일부를 가리고 다음 단어를 맞추며 언어를 배우듯, 데이터 자체에서 자동으로 레이블을 만들어 학습합니다.

### 강화학습

게임을 하면서 점수를 높이는 방법을 터득하듯, 시행착오를 통해 보상을 최대화하는 행동을 학습합니다.

## ▶ 지도학습

입력과 정답(레이블)이 쌍으로 주어진 데이터로 학습.

마치 선생님이 문제와 답을 함께 알려주며 가르치는 것과 같습니다.

### 대표과제 (분류/회귀)

- **분류**: 이메일이 스팸인지 아닌지 판단
- **회귀**: 주택 가격 예측, 매출 예측

### 예시 데이터 셋

이미지:  → 레이블: "고양이"

이미지:  → 레이블: "강아지"

이미지:  → 레이블: "여우"

## 학습 패러타임 변화\_어떻게 학습할 것인가?

### ▶ 비지도학습

정답 없이 데이터의 숨겨진 패턴을 스스로 발견

책장에 있는 책들을 장르별로 자동으로 분류하기 (장르 표시 없음)

**대표과제 (분류/회귀)**

- 군집화 (Clustering): 고객을 구매 패턴에 따라 그룹화
- 차원축소 (Dimensionality Reduction): 복잡한 데이터를 2D로 시각화

**예시 시나리오**

데이터: 고객 1000명의 구매 내역

목표: 비슷한 구매 패턴을 가진 고객끼리 묶기

결과: "명품 선호 그룹", "가성비 중시 그룹", "트렌드 추종 그룹" 등

## 학습 패러다임 변화\_어떻게 학습할 것인가?

### ▶ 자기지도학습

데이터 자체에서 자동으로 레이블을 만들어 학습

책의 일부를 가리고 다음 단어 맞추기 게임을 스스로 하며 언어를 배우기

### 학습방식

**Masked Learning:** "오늘 [MASK] 매우 좋다" → "날씨가" 예측

**Contrastive Learning:** 같은 고양이 사진(다른 각도)은 비슷함  
고양이 vs 강아지는 다름으로 구별

### 대표 모델

BERT (언어), ViT (비전), GPT (언어)



## 학습 패러다임 변화\_어떻게 학습할 것인가?

### ▶ 강화학습

시행착오를 통해 보상을 최대화하는 행동을 학습합니다.  
게임을 하면서 점수를 높이는 방법을 스스로 터득하는 것과 같습니다.

#### 핵심 요소

- 상태 (State): 현재 게임 화면
- 행동 (Action): 위/아래/좌/우 이동
- 보상 (Reward): 적 제거 +10점, 사망 -100점

#### 예시

로봇 청소기가 수천 번의 시행착오 후 최적 청소 경로를 스스로 학습

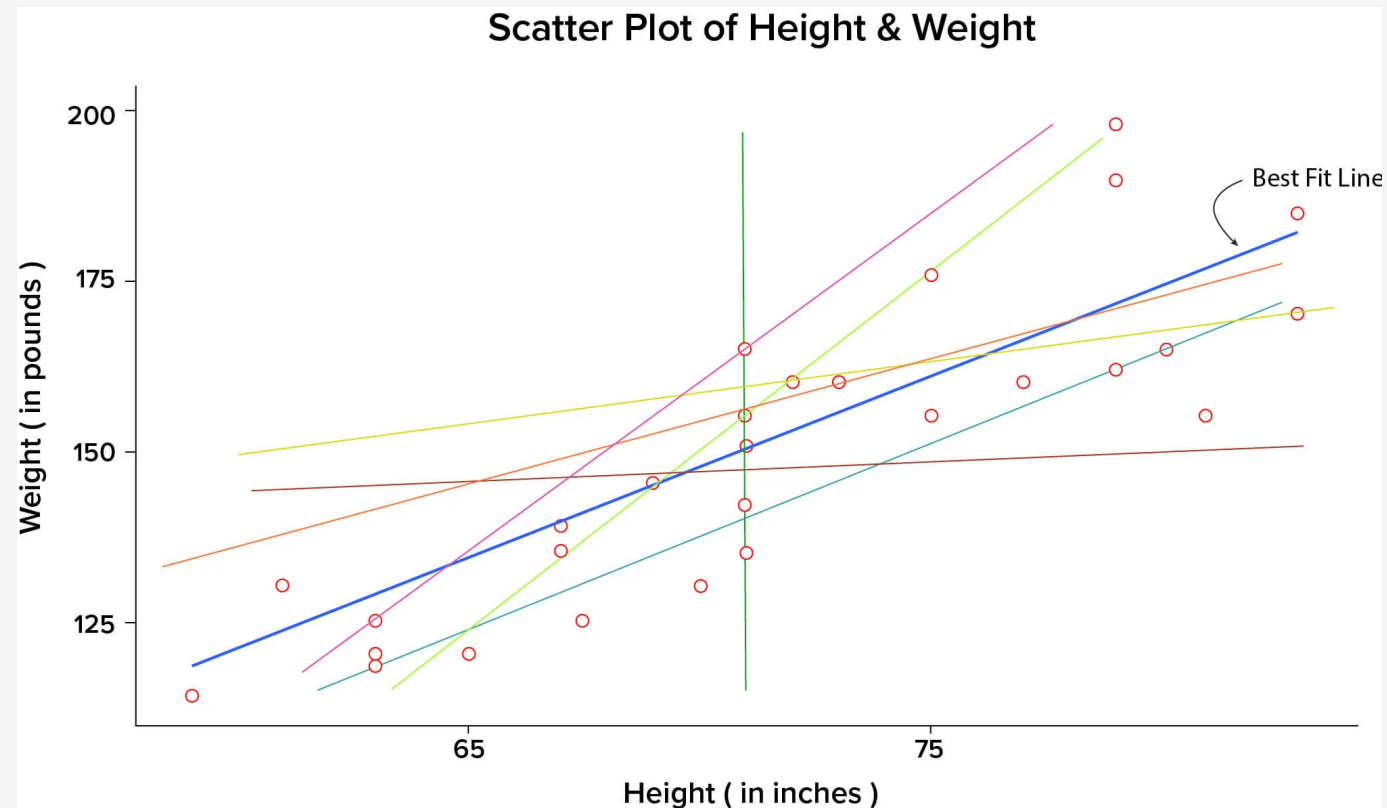
## ▶ 강화학습

### 로봇 청소기 학습

- 먼지 제거 성공 → +1점
  - 장애물 충돌 → -5점
  - 배터리 부족 전 충전소 도착 → +10점
- 수천 번 시행착오 후 최적 청소 경로 학습

- 모든 딥러닝의 시작은 '선형 모델'
- 가장 단순하지만, 모든 복잡한 신경망의 기본 단위
- 입력(X)과 가중치(W)의 선형결합(Linear Combination)

$$y = wx + b$$



## 문제 정의: 신장으로 체중 예측하기

머신러닝의 첫걸음은 우리가 풀고 싶은 문제를 명확히 정의하는 것입니다. 다섯 명의 신장과 체중 데이터가 주어졌을 때, 이 관계를 학습하여 새로운 신장 값이 주어지면 그 사람의 체중을 예측하는 모델을 만드는 것입니다.

# 166

첫 번째 데이터

신장 166cm, 체중 58.7kg

# 176

두 번째 데이터

신장 176cm, 체중 75.7kg

# 171

세 번째 데이터

신장 171cm, 체중 62.1kg

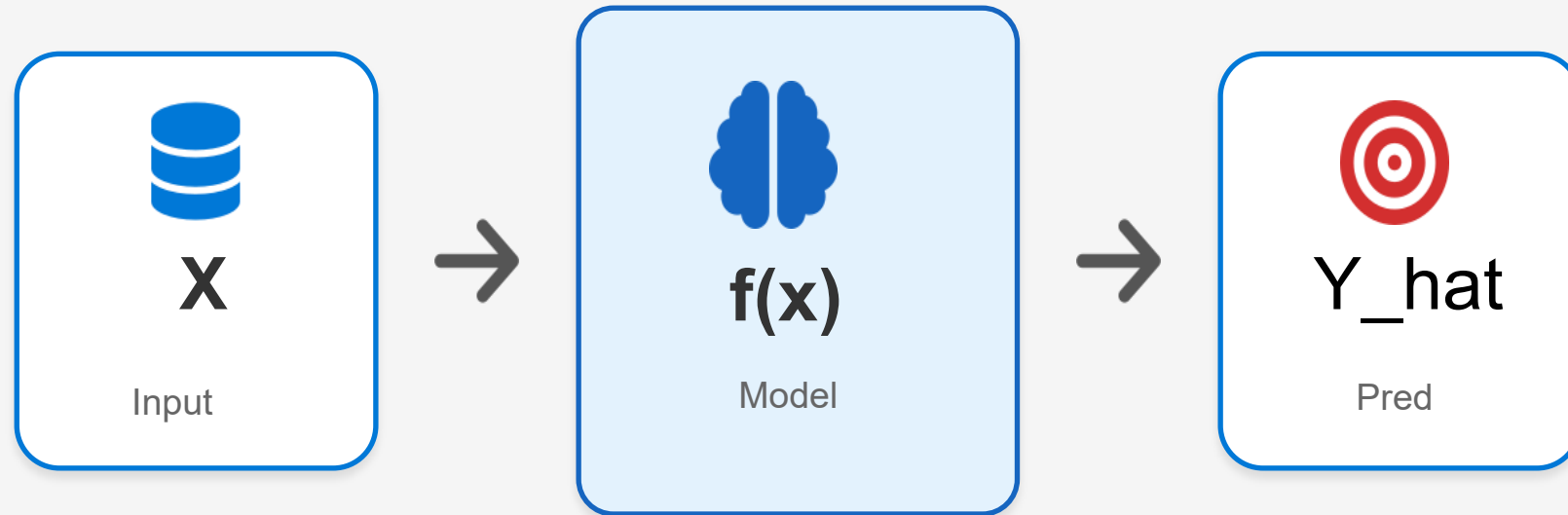
우리의 목표는 이 점들을 가장 잘 대표하는 하나의 직선을 찾는 것입니다. 이것이 가장 단순한 형태의 머신러닝 모델입니다.

## 왜 선형회귀로부터 배우는가?

- 단순하지만 모든 신경망의 가장 기본이 되는 블록입니다.
- 복잡한 모델도 결국 모듈화와 파라미터 학습의 확장일 뿐입니다.
- 최신 모델(CNN, Transformer 등)도 이 구조를 반복해서 쌓은 형태입니다.
- 즉, **"CNN, Transformer도 결국 이 구조의 반복입니다."**

## 머신러닝 첫 단계 : 문제 정의

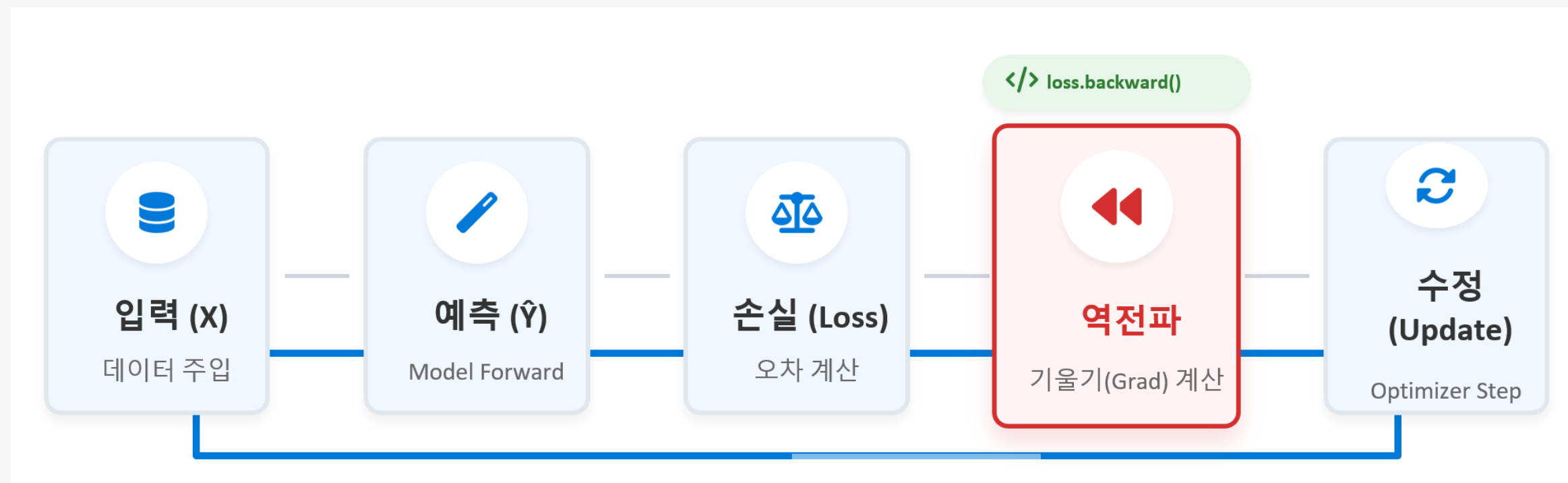
- 입력(X)와 정답(Y)의 명확한 정의가 가장 우선
- 예측 함수의 목적과 최종 출력 형태 결정
- 과업(Task)의 정의가 곧 데이터와 평가지표를 결정



입력 데이터를 모델에 통과시켜 예측값을 얻는 과정

- 핵심

모든 단계는 PyTorch 코드의 `for epoch in range():` 루프 안에서 순차적으로 실행



우리가 앞으로 배울 모든 복잡하고 화려한 딥러닝 모델의 심장에는 아주 단순하지만 강력한 원리가 자리 잡고 있습니다.

바로 '틀린 부분을 조금씩 고쳐나가며 정답에 가까워지는 것'이죠.

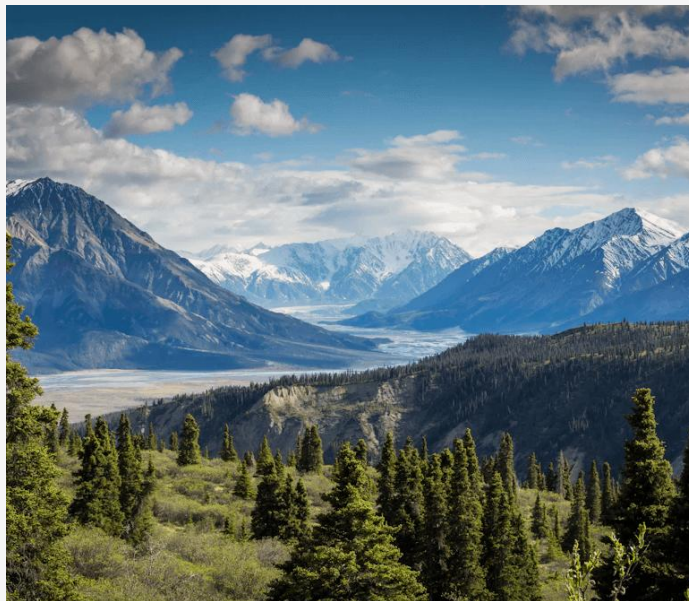
선형 회귀와 경사 하강법은 머신러닝의 '자전거 타기'와 같습니다.

'신장으로 체중 예측하기'라는 단순한 문제를 통해

모델이 어떻게 머신러닝이 적용되는지 확인해보겠습니다.



## 경사 하강법: 안갯속에서 산 내려오기



01

### 현 위치 탐색

동서남북으로 1미터씩 이동해보고  
각각의 고도 변화를 측정합니다.

02



### 최적 방향 결정

고도가 가장 가파르게 높아지는 방향을  
찾습니다.

03

### 이동 및 반복

가장 가파른 방향으로 일정 거리를  
이동하고 과정을 반복합니다.

## 경사하강법 비유: 산 내려오기

- 현재 고도  $\leftrightarrow$  손실(Loss)
    - 가장 낮은 지점(Global Minimum)을 찾아 내려가는 것이 목표
  - 발의 기울기  $\leftrightarrow$  그라디언트(Gradient)
    - 어느 방향으로 이동할 지 결정(가파른 곳 일수록 빠르게 이동)
  - 보폭(Step)  $\leftrightarrow$  학습률(Learning Rate)
    - 한 번에 얼마나 이동할지 결정(너무 크면 지나치고, 작으면 느림)
- 
- 정답을 한 번에 찾는 것이 아니라, 손실을 줄이는 방향으로 이동합니다.
  - 전체 지도를 몰라도 현재 위치의 기울기(지역 정보)만으로 전진할 수 있습니다.
  - 학습률(Learning Rate)에 따라 수렴 속도와 안정성이 결정됩니다.

“

**"최적의 파라미터는 계산하는 것이 아니라 찾아가는(Searching) 것입니다."**

이 반복적인 탐색 과정이 바로 기계 '학습'의 핵심입니다.

### 핵심 포인트



우리가 집중해야 할 '역전파(Backprop)'는 전체 학습 과정 중 3단계에 해당하며, 손실을 줄이기 위해 각 파라미터가 어느 방향으로 움직여야 할지(기울기)를 계산하는 과정입니다.

# 경사 하강법의 목표와 4단계



# 역전파(Backpropagation)란 무엇인가?

## CORE DEFINITION

"출력의 오류(Error)를 입력 방향으로 되돌리며  
각 파라미터의 책임을 나누는 과정"



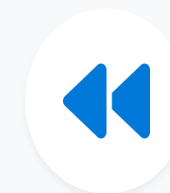
### 연쇄 법칙 (Chain Rule)

복잡한 미분을 단계별로 쪼개어  
곱셈으로 연결하는 수학적 원리



### 기여도 계산

손실(Loss) 발생에 대해  
각 가중치가 얼마나 기여했는지 계산



### 역방향 흐름

결과(Output)에서 시작하여  
원인(Input) 방향으로 정보 전달



### 결과는 하나, 원인은 여러 개

최종 오차(Loss)는 단일 값이지만, 이를 유발한 원인(가중치)은 수만 개 이상 존재합니다.



### 출력에서 시작해야 영향 경로 분명

결과에서 입력 방향으로 거슬러 올라가야 각 파라미터가 오차에 미친 기여도를 효율적으로 계산할 수 있습니다.



### 계산 그래프에서 경로별 기여 합산

연쇄 법칙(Chain Rule)을 통해 그래프의 각 노드별 미분값을 곱하며 입력층까지 그라디언트를 전파합니다.

```
train_step.py

# 1. 모델 예측 (Forward)
y_pred = model(x_train)

# 2. 손실 계산
loss = criterion(y_pred, y_true)

# 3. 역전파 수행
loss.backward()

# 4. 가중치 갱신
optimizer.step()
```

### 자동 미분 (Autograd)

계산 그래프를 역순으로 탐색하며  
연쇄 법칙(Chain Rule)에 따라 기울기를  
자동으로 누적합니다.

### .grad 속성에 저장

계산된 모든 기울기는  
각 파라미터 텐서의 .grad 속성에 저장됩니다.

### 3차시 연결 Recall

지난 시간 실습한 `requires_grad=True` 설정이  
바로 이 과정을 가능하게 하는 스위치입니다.

## 활성화 함수가 없다면?

- 선형 함수의 합성은 여전히 선형  $\text{Linear} + \text{Linear} = \text{Linear}$  (수학적 사실)
- 층을 쌓아도 하나의 직선에 불과 깊은 신경망(Deep Network)의 효과가 사라짐
- 표현력 부족으로 복잡한 패턴 학습 불가 이미지, 자연어 같은 비선형 데이터를 처리할 수 없음

### ⚠ 결론: "Deep" Learning이 불가능

활성화 함수 없이는 아무리 많은 뉴런과 층을 연결해도,  
결국 단 하나의 선형 회귀 모델과 수학적으로 동일합니다.



- **비선형성(Non-linearity) 부여**
  - 선형 변환(행렬 곱)만으로는 불가능한 모델의 표현력(Representation Power)을 획기적으로 증가시킵니다.
- **복잡한 결정 경계 학습 가능**
  - 단순한 직선이 아닌, 데이터의 복잡한 패턴을 구분할 수 있는 곡선 형태의 경계를 만들어냅니다.
- **학습 안정성과 수렴에 영향**
  - 올바른 함수 선택은 기울기 소실(Vanishing Gradient) 문제를 완화하고, 최적해로의 수렴 속도를 결정합니다.

“활성화 함수가 없다면, 아무리 깊은 신경망(Deep Network)을 쌓아도 수학적으로는 결국 하나의 층(Single Layer)과 동일해집니다.”

 **Sigmoid ( $\sigma$ )**

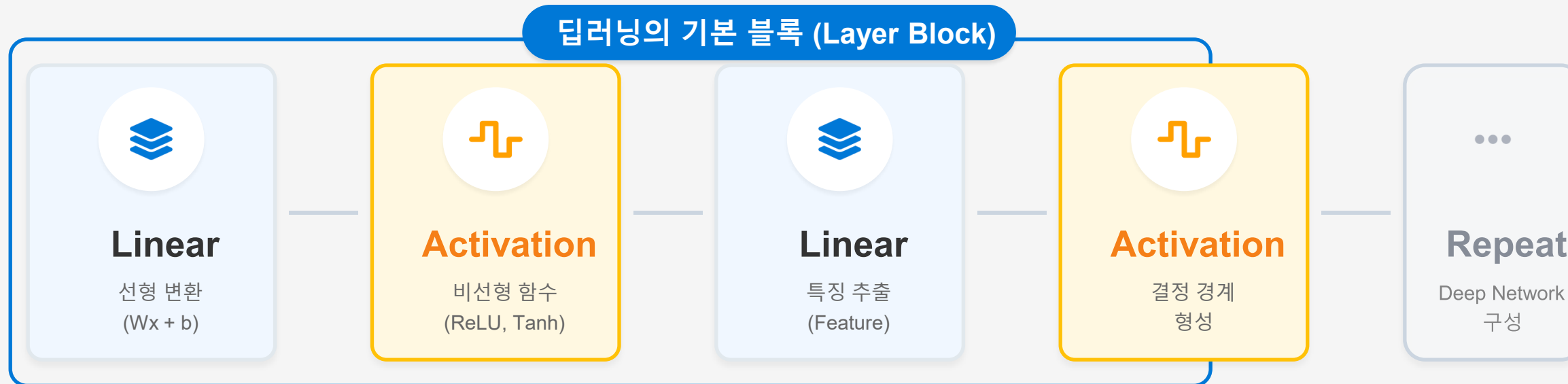
- 확률적 해석 가능 (0~1 사이 출력)
- 치명적 단점: 양 끝단 포화 영역에서 기울기 소실(Vanishing Gradient) 발생

 **Tanh (Hyperbolic Tangent)**

- 출력 중심화(Zero-centered): -1~1 사이 출력, 최적화 효율↑
- 여전히 양 끝단에서 기울기 소실 문제 존재

 **ReLU (Rectified Linear Unit)**

- 희소성(Sparsity) 확보 및 연산 효율성 우수
- 양수 영역에서 기울기 소실 완화 / 주의: Dying ReLU (음수 입력 시 뉴런 사망)



### 모든 최신 모델의 공통 DNA



이 Linear + Activation의 반복 구조는 MLP(다층 퍼셉트론), CNN(합성곱 신경망), Transformer(LLM의 기반) 모두에 공통적으로 적용됩니다.

머신러닝 모델이 학습하기 좋은 형태로 데이터를 가공하는 **전처리 과정**을 거칩니다.

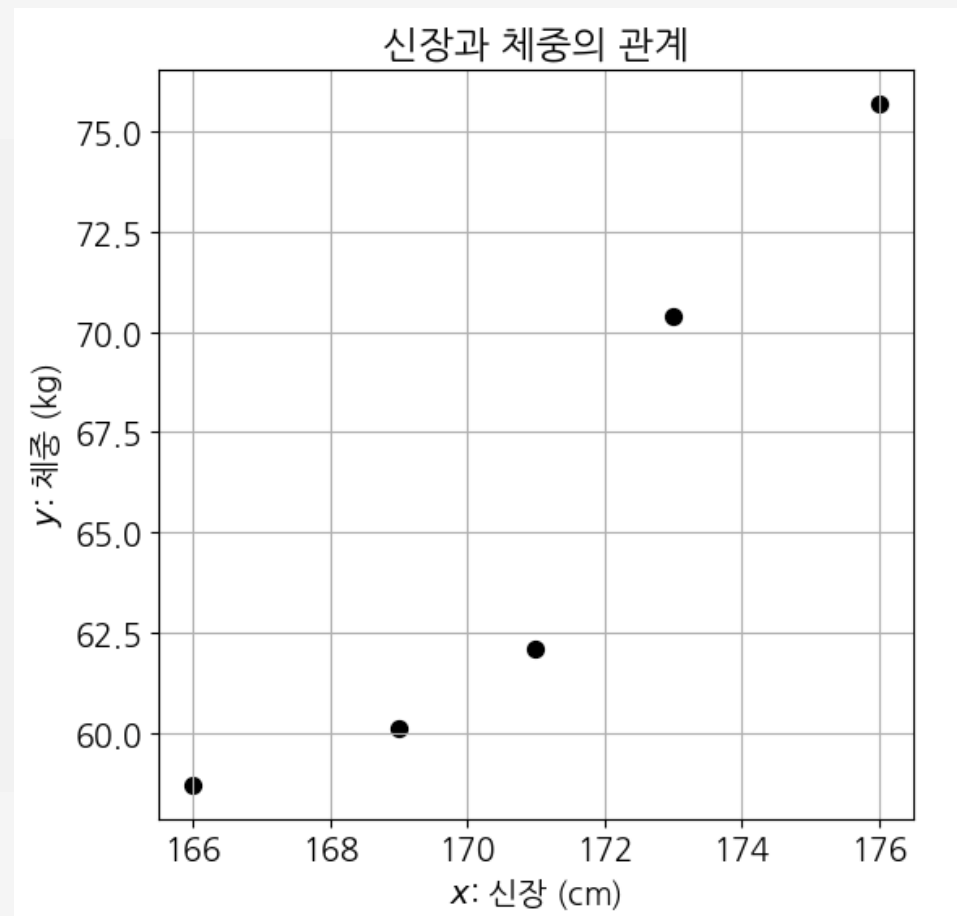
```
# 샘플 데이터 선언 (신장, 체중)
sampleData1 = np.array([
    [166, 58.7],
    [176.0, 75.7],
    [171.0, 62.1],
    [173.0, 70.4],
    [169.0, 60.1]
])
```

numpy 라이브러리를 사용해 5명의 신장과 체중 데이터를 2차원 배열로 선언했습니다.

학습 데이터를 입력 데이터  $x$ (신장)와 정답 데이터  $y$ (체중)로 분리합니다.

```
# 머신러닝 모델에서 사용하기 위해, 신장을 변수 x로, 체중을 변수 y로 함
x = sampleData1[:,0] # 모든 행의 0번째 열(신장)을 x로 지정
y = sampleData1[:,1] # 모든 행의 1번째 열(체중)을 y로 지정
# 산포도 출력 확인
plt.show()
```

`sampleData1[:,0]`은 NumPy의 슬라이싱 기능으로, 모든 행의 첫 번째 열 데이터를 선택합니다.



경사 하강법은 데이터의 절댓값이 1을 넘지 않는 비교적 작은 값일 때 더 안정적으로 작동합니다.

각 데이터에서 평균값을 빼주어 데이터의 중심을 0으로 맞춥니다.

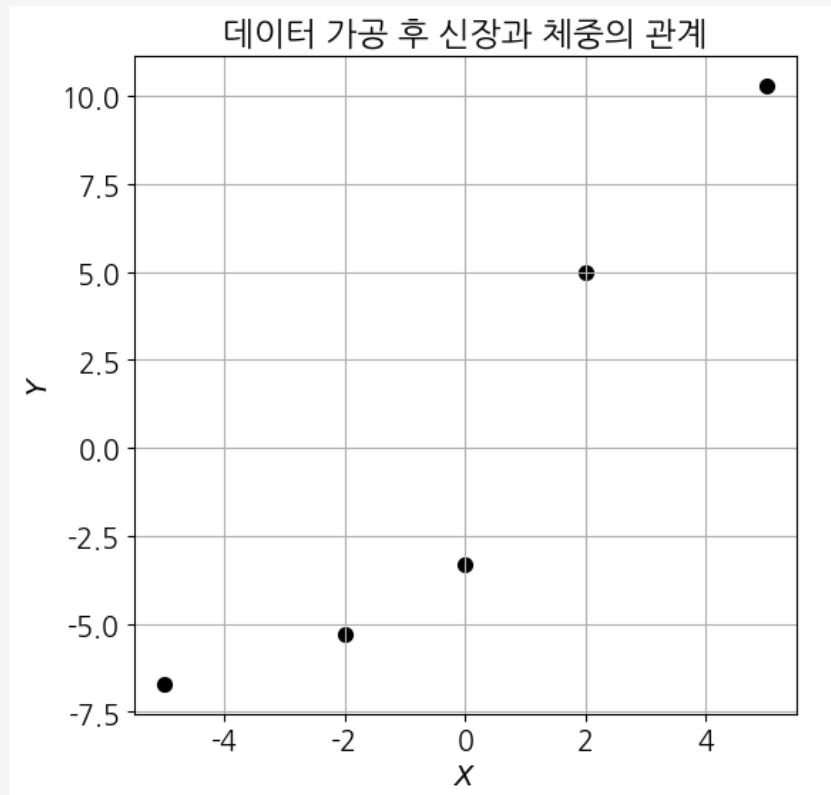
```
X = x - x.mean() # x의 모든 요소에서 x의 평균값을 뺌
```

```
Y = y - y.mean() # y의 모든 요소에서 y의 평균값을 뺌
```

### 변환 효과

값의 범위가 0을 중심으로 훨씬 작아졌습니다.

데이터의 상대적인 관계는 그대로 유지됩니다.



우리가 찾고자 하는 1차 함수의 계수  $W$ 와 절편  $B$ 를 텐서 변수로 정의합니다.

```
# 파라미터 정의
# W와 B는 경사 계산을 위해, requires_grad=True로 설정함
W = torch.tensor(1.0, requires_grad=True).float()
B = torch.tensor(1.0, requires_grad=True).float()

# 예측 함수는 1차 함수
def pred(X):
    return W * X + B
```

❏ 중요: `requires_grad=True` 속성을 `True`로 설정해야만 파이토치의 자동 미분 기능이  $W$ 와  $B$ 에 대한 모든 연산을 추적하고, 나중에 경사를 자동으로 계산할 수 있습니다.

경사 하강법에서 예측 다음 단계는 손실 계산입니다. 우리는 손실 함수로 MSE(평균 제곱 오차)를 사용합니다.

### MSE 함수 정의

```
def mse(Yp, Y):  
    loss = ((Yp - Y)**2).mean()  
    return loss
```

### 손실 계산

```
# 손실 계산  
loss = mse(Yp, Y)  
  
# >>> tensor(16.5980, grad_fn=<MeanBackward0>)
```

mse 함수는 예측값  $Y_p$ 와 실제값  $Y$ 의 차이를 제공한 뒤, 모든 데이터에 대해 평균을 내어 손실 값을 계산합니다.



손실까지 계산했다면, 경사 계산은 놀라울 만큼 간단합니다. loss 텐서에 대해 .backward() 함수를 호출하기만 하면 됩니다.

```
# 경사 계산
loss.backward()
# 경사값 확인
print(W.grad) # >>> tensor(3.1600)
print(B.grad) # >>> tensor(2.6800)
```

loss.backward()가 호출되면, 파이토치는 loss부터 시작해서 계산 그래프를 거꾸로 거슬러 올라가며 requires\_grad=True로 설정된 모든 텐서에 대한 경사를 계산하여 각 텐서의 .grad 속성에 저장합니다.

계산된 경사값을 이용해 W와 B를 수정할 차례입니다. 경사 하강법의 기본 원리는 현재 파라미터 값에서 학습률  $\times$  경사값을 빼주는 것입니다.

## 잘못된 방법 (오류 발생)

```
lr = 0.001 # 학습률 정의
# 경사를 기반으로 파라미터 수정
W -= lr * W.grad
B -= lr * B.grad
```

RuntimeError: a leaf Variable that requires grad is being used in an in-place operation.

## 올바른 방법

```
with torch.no_grad():
    # 경사를 기반으로 파라미터 수정
    W -= lr * W.grad
    B -= lr * B.grad
# 계산이 끝난 경사값을 초기화함
W.grad.zero_()
B.grad.zero_()
```

with torch.no\_grad(): 블록 내에서는 계산 그래프 생성이 일시적으로 중단되어 안전하게 수정 가능

지금까지의 과정을 합쳐서 반복 계산을 수행합니다.

```
# 초기화
# W와 B를 변수로 사용
W = torch.tensor(1.0, requires_grad=True).float()
B = torch.tensor(1.0, requires_grad=True).float()

# 반복 횟수
num_epochs = 500

# 학습률
lr = 0.001

# history 기록을 위한 배열 초기화
history = np.zeros((0, 2))
```

```
# 루프 처리
for epoch in range(num_epochs):
    Yp = pred(X)      # 예측 계산
    loss = mse(Yp, Y) # 손실 계산
    loss.backward()   # 경사 계산

    # 경사를 기반으로 파라미터 수정
    with torch.no_grad():
        W -= lr * W.grad
        B -= lr * B.grad

    # 경사값 초기화
    W.grad.zero_()
    B.grad.zero_()
```

반복 계산이 끝난 후 우리가 찾은 최종 파라미터  $W$ ,  $B$ 와 학습 전후의 손실 값이 어떻게 변했는지 확인합니다.

**1.82****최종 가중치  $W$** 

학습을 통해 얻은 기울기 값

**0.37****최종 편향  $B$** 

학습을 통해 얻은 절편 값

**13.35****초기 손실**

학습 시작 시점의 오차

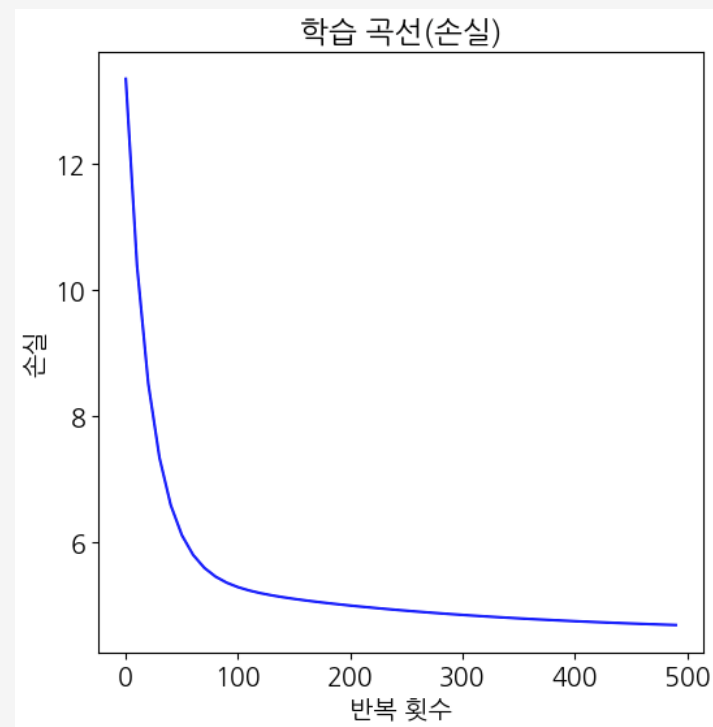
**4.68****최종 손실**

학습 완료 후의 오차

손실이 13.35에서 4.67로 크게 줄어든 것을 수치적으로 확인할 수 있습니다.

숫자로만 보면 손실이 어떻게 줄어들었는지 과정이 잘 보이지 않습니다. matplotlib 라이브러리를 사용해 '학습 곡선'을 그려서 변화를 한눈에 파악해 봅시다.

```
# 학습 곡선 출력(손실)
# X축: history의 모든 행의 첫 번째 열(반복 횟수)
# Y축: 모든 행의 두 번째 열(손실)
plt.plot(history[:,0], history[:,1], 'b')
plt.show()
```



그래프에서 볼 수 있듯이, 학습 초기에 손실이 급격하게 감소하다가 점차 완만하게 수렴하는 아름다운 곡선이 나타납니다. 이는 경사 하강법을 통한 학습이 성공적으로 진행되었음을 의미합니다.

## ▶ 국소 최적화 문제

모델을 학습시킨다는 것은, 마치 안개 속에서 산을 내려오는 등산가와 같습니다.  
우리의 목표는 가장 낮은 해발고도, 즉 '가장 적은 오류'를 가진 지점을 찾는 것이죠.



### 전역 최적해

산 전체에서 가장 낮은 계곡으로, 진짜 최고의 답을 의미합니다.



### 국소 최적해

산 중턱에 움푹 파인 작은 함정으로, 꽤 괜찮지만 최고는 아닌 답입니다.

만약 우리가 내려온 곳이 국소 최적해라면, 그 함정 안에서는 사방이 오르막길이라 우리는 그곳이 가장 낮은 지점이라고 착각하고 탐험을 멈출지 모릅니다.

### ▶ 국소 최적화 문제

- 초기 파라미터(Weight) 값에 따라 서로 다른 지점으로 수렴하는 문제가 발생합니다
- 손실 함수 지형이 울퉁불퉁한 비볼록(Non-convex) 형태라, 곳곳에 함정이 존재합니다.
- 단순 경사하강법은 진짜 최소점(Global Minimum)이 아닌 국소 최적해(Local Minimum) 갇힐 수 있음

“

**"어디서 출발하느냐가 최종 도착지를 결정합니다."**

항상 가장 낮은 곳(Global Minimum)으로 갈 수 있다는 보장이 없습니다.

## 왜 최적화가 필요한가?

- 단순 경사하강법(GD)의 한계: 관성 부재, 진동, 느린 수렴 등 학습 효율 저하 문제가 발생합니다.
- 학습 속도와 안정성을 높이기 위해 적응형 학습률과 모멘텀이 필요합니다.

“

**"단순한 하강만으로는 복잡한 지형을 탈출하기 어렵습니다."**

더 스마트한 도구(Optimizer)가 학습의 성공을 좌우합니다.



## 왜 최적화가 필요한가?

### ▶ 최적화 함수(Optimizer) 소개

지금까지  $W -= lr * W.grad$ 와 같이 직접 파라미터 값을 수정했지만, 이제 파이토치의 최적화 함수를 사용해봅시다.  
코드가 훨씬 간결하고 강력해집니다.



#### 전문 코치의 역할

최적화 함수는 파라미터 업데이트 과정을 자동화하고 더 지능적으로 수행하는 전문 코치입니다.



#### 간편한 사용법

`torch.optim` 라이브러리에서 SGD와 같은 코치를 불러와 파라미터들을 맡기기만 하면 됩니다.



#### 자동 업데이트

`optimizer.step()`이라는 구령 한 번에 모든 파라미터들이 알아서 다음 훈련 단계로 나아갑니다.

## 최적화 함수의 구현

파이토치의 최적화 함수를 사용한 학습 코드를 구현해봅시다.

```
# 초기화
# W와 B를 변수로 사용
W = torch.tensor(1.0, requires_grad=True).float()
B = torch.tensor(1.0, requires_grad=True).float()

# 반복 횟수
num_epochs = 500
# 학습률
lr = 0.001

# optimizer로 SGD(확률적 경사 하강법)을 사용
import torch.optim as optim # optim 라이브러리 import
# SGD 옵티마이저 생성. 관리할 파라미터([W, B])와 학습률(lr)을 알려줌
optimizer = optim.SGD([W, B], lr=lr)

# history 기록을 위한 배열 초기화
history = np.zeros((0, 2))
```

```
# 루프 처리
for epoch in range(num_epochs):
    # 예측 계산
    Yp = pred(X)

    # 손실 계산
    loss = mse(Yp, Y)

    # 경사 계산
    loss.backward()

    # 파라미터 수정
    optimizer.step() # 옵티마이저가 알아서 W와 B를 업데이트함

    # 경사값 초기화
    optimizer.zero_grad() # 옵티마이저에 연결된 파라미터들의 경사도를
    0으로 초기화
```

optimizer.step()과 optimizer.zero\_grad() 두 함수만으로 복잡한 파라미터 업데이트 과정을 간단하게 처리할 수 있습니다.

최적화 함수의 진정한 힘은 '튜닝'에 있습니다.

SGD에 momentum이라는 옵션을 추가하여 학습 속도가 어떻게 변하는지 비교해 보겠습니다.

## 기본 SGD

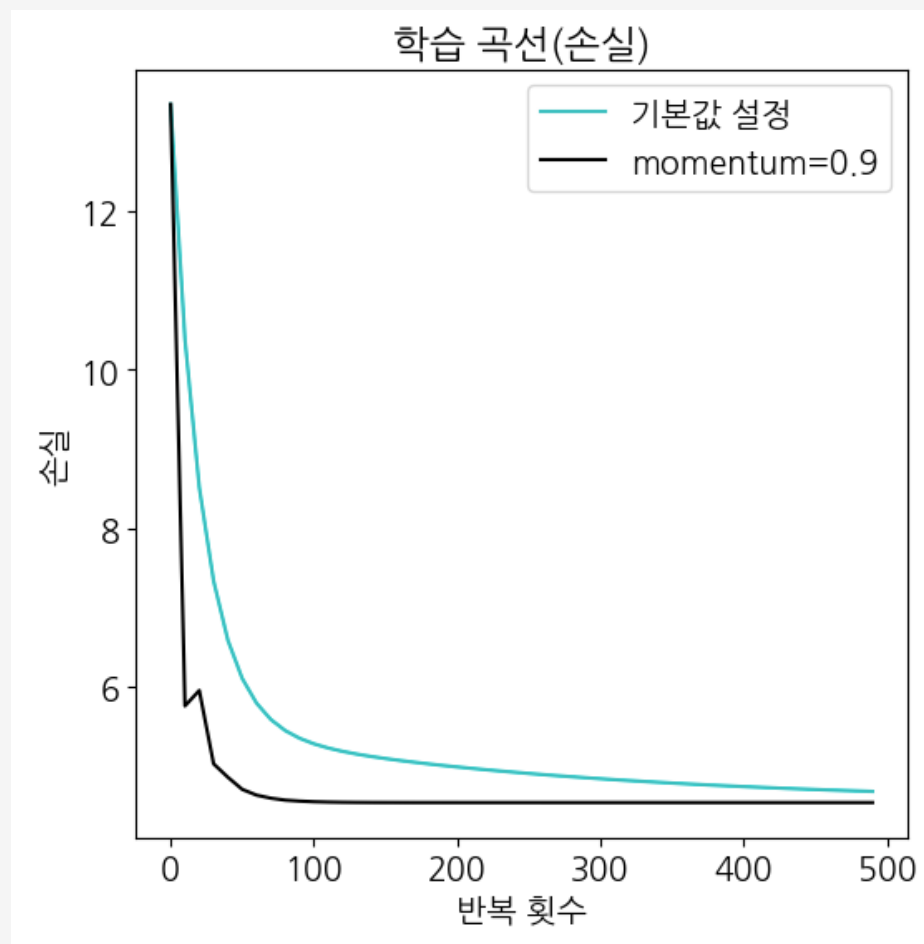
```
optimizer = optim.SGD([W, B], lr=lr)
```

기본적인 확률적 경사 하강법

## Momentum 적용

```
optimizer = optim.SGD([W, B], lr=lr,  
momentum=0.9)
```

**momentum=0.9 옵션으로 학습 가속화**



momentum 알고리즘이 학습을 가속화하여 더 적은 반복만으로도 낮은 손실에 도달하게 해줍니다.

복잡한 이론을 직접 구현할 필요 없이 파라미터 변경만으로 간단하게 학습 알고리즘을 튜닝하고 성능을 개선할 수 있습니다.

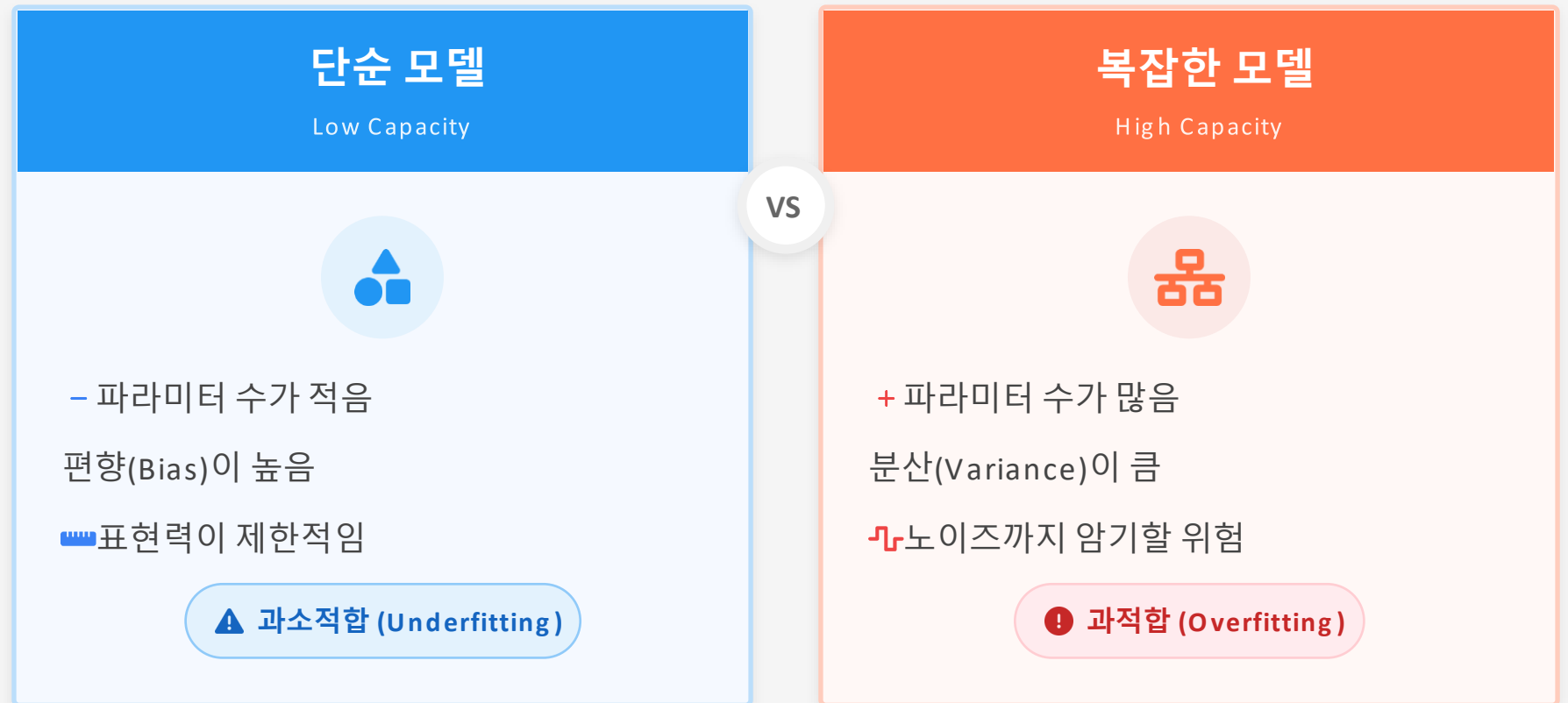
모델이 학습할 수 있는 복잡한 패턴의 범위. 마치 공부능력과 같음

낮은 용량:

- 초등학생
- 간단한 덧셈만 가능

높은 용량:

- 대학생
- 미적분까지 가능



목표: 적절한 균형점(Sweet Spot) 찾기

## 편향-분산 트레이드오프(Bias-Variance Trade-off)

### 편향 (Bias)

모델이 너무 단순해서 패턴을 제대로 학습하지 못하는 오류입니다. 예: "키가 크면 무조건 남자"라고 과도하게 일반화하는 것.

결과: 과소적합 (Underfitting)

### 분산 (Variance)

모델이 너무 복잡해서 훈련 데이터의 노이즈까지 학습하는 오류입니다. 예: "170.3cm는 남자, 170.2cm는 여자"처럼 과도하게 세밀한 판단.

결과: 과대적합 (Overfitting)

# 편향-분산 트레이드오프(Bias-Variance Trade-off)

## 과소적합 (Underfitting)

데이터 패턴을 학습하지 못함



모델이 지나치게 단순함

Train / Validation 오차 모두 높음

편향 (Bias)이 매우 높음

⚠ High Bias

VS

## 과적합 (Overfitting)

노이즈까지 암기해버림



훈련 데이터에 너무 민감함

Train 오차 낮음 / Val 오차 높음

분산 (Variance)이 매우 높음

⚠ High Variance

🔍 훈련 성능과 검증 성능의 차이(Gap) 관찰

해결책: Dropout, 정규화(L1/L2), 데이터 추가 등

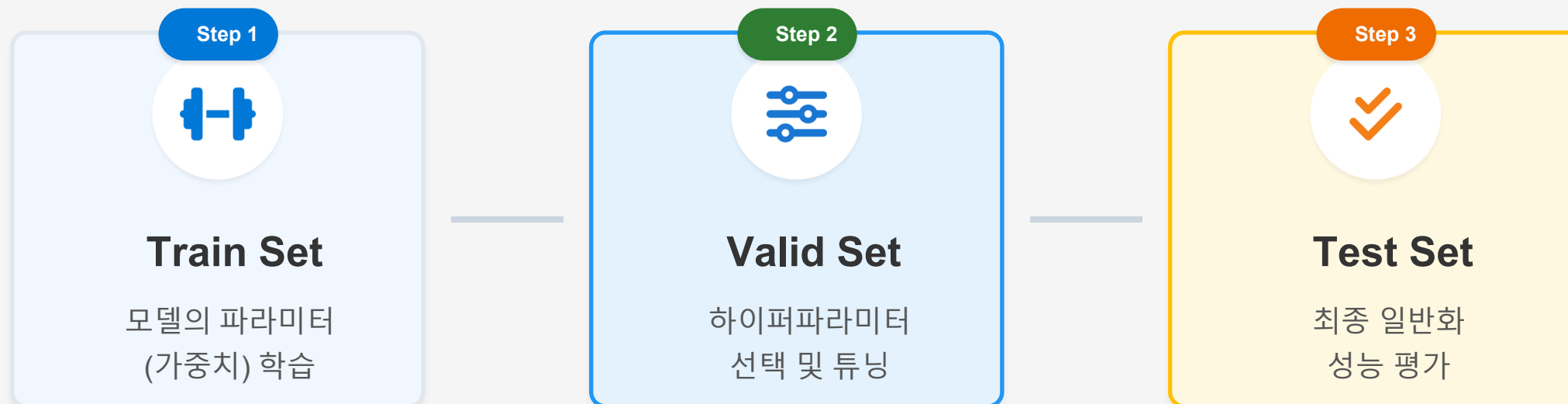
▶ 왜 데이터를 나눌까?

시험 준비하기

훈련 세트: 교과서로 공부하기

검증 세트: 모의고사로 실력 체크

테스트 세트: 실제 수능 시험





### 핵심 : 데이터 누수 (Data Leakage) 방지

- 시험 문제(Test)를 미리 보고 공부(Train)하면 실력을 제대로 평가할 수 없습니다.
- 검증(Valid)과 평가(Test) 데이터는 학습 과정에 절대 직접 관여해서는 안 됩니다.

## 표준 분할 비율



## 대용량 데이터

(10만 개 이상)

- Train: 80%
- Validation: 10%
- Test: 10%



## 중소규모 데이터

(1만 개 이하)

- Train: 60%
- Validation: 20%
- Test: 20%



## 소량 데이터

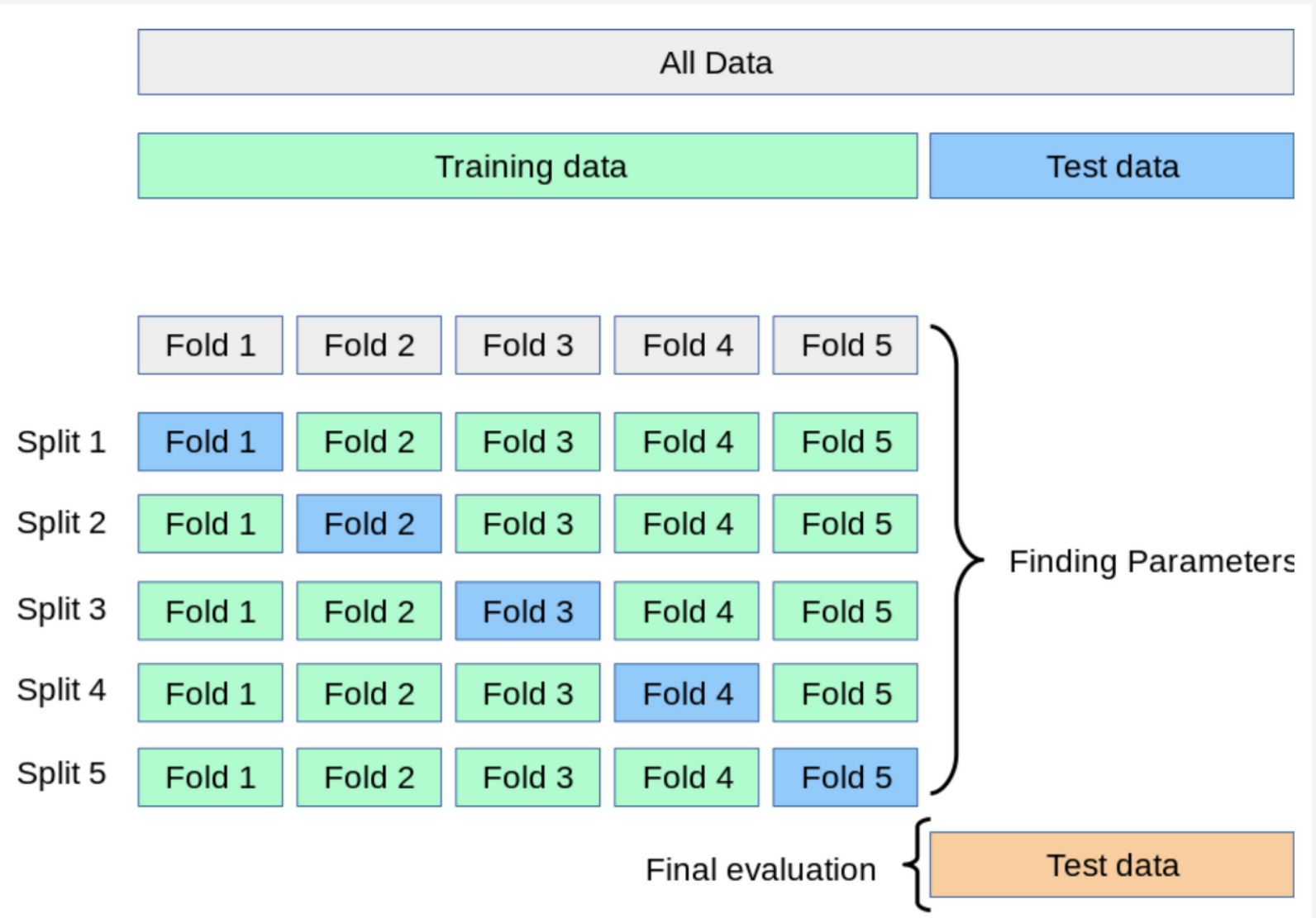
(1천 개 이하)

K-Fold 교차검증 사용 권장

### ▶ K-FOLD 교차검증

- 데이터를 K개로 나누고, 돌아가면서 검증 세트로 사용하는 방식
- 적은 데이터도 충분히 활용할 수 있는 장점이 있습니다.

**5번의 성능 평균  
= 최종 성능**



## ▶ 모델을 어떻게 평가할까요?

## 지표 선택 가이드

| 상황      | 1차 지표     | 보조 지표             |
|---------|-----------|-------------------|
| 회귀 (일반) | MAE       | RMSE, $R^2$       |
| 균형 분류   | Accuracy  | F1-Score          |
| 불균형 분류  | F1-Score  | Precision, Recall |
| 의료 진단   | Recall    | F1-Score          |
| 스팸 필터   | Precision | F1-Score          |

## 회귀 (Regression)

연속된 수치 예측



MSE (Mean Squared Error)

MAE (Mean Absolute Error)

$R^2$  Score (결정 계수)

🎯 오차(Error) 최소화 목표

VS

## 분류 (Classification)

카테고리/클래스 판별



Accuracy (정확도)

Precision & Recall

F1 Score (조화 평균)

✔️ 정답 매칭률 최대화



### MAE (평균 절대 오차)

해석이 가장 직관적입니다. "평균적으로 13.33만큼 틀렸다"처럼 바로 이해할 수 있어 비즈니스 보고용으로 최적입니다.



### RMSE (평균 제곱근 오차)

큰 오차에 더 큰 페널티를 줍니다. 원래 단위로 복구되어 MAE처럼 해석 가능하며, 큰 오차를 더 중요하게 볼 때 사용합니다.



### $R^2$ (결정계수)

0~1 범위의 값으로 모델의 설명력을 나타냅니다. 0.95면 변동의 95%를 설명한다는 의미입니다.

카테고리를 예측하는 문제에 사용

- 정확도
- 정밀도
- 재현율
- F1-SCORE
- ROC-AUC

## 성능 평가 지표 : 혼동행렬

- 분류 모델의 성능을 시각화하여 평가하기 위한 표
- 실제 값과 모델의 예측 값을 비교하여 모델이 얼마나 정확하게 분류하는지 보여줍니다

|                 |   | Predicted category |          |
|-----------------|---|--------------------|----------|
|                 |   | A                  | B        |
| Actual category | A | <b>O</b>           | <b>X</b> |
|                 | B | <b>X</b>           | <b>O</b> |

|                 |   | Predicted category |           |
|-----------------|---|--------------------|-----------|
|                 |   | A                  | B         |
| Actual category | A | <b>TP</b>          | <b>FN</b> |
|                 | B | <b>FP</b>          | <b>TN</b> |



### 정확도 (Accuracy)

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{전체} = (50 + 35) / 100 = 0.85 \text{ (85\%)}$$

- 전체 중 맞춘 비율
- 불균형 데이터에서는 쓸모없음  
예: 암 환자 1%, 정상 99% → 전부 정상이라고 해도 99% 정확도

### 정밀도 (Precision)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 50 / (50 + 5) = 0.91 \text{ (91\%)}$$

- **질문**: "양성이라고 예측한 것 중 실제 양성 비율은?"
- **중요한 상황**: 스팸 필터 (정상 메일을 스팸으로 잘못 분류하면 안 됨)

재현율 / 민감도 (Recall)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 50 / (50 + 10) = 0.83 \text{ (83\%)}$$

- 질문: "실제 양성 중 몇 개를 찾아냈나?"
- 중요한 상황: 암진단

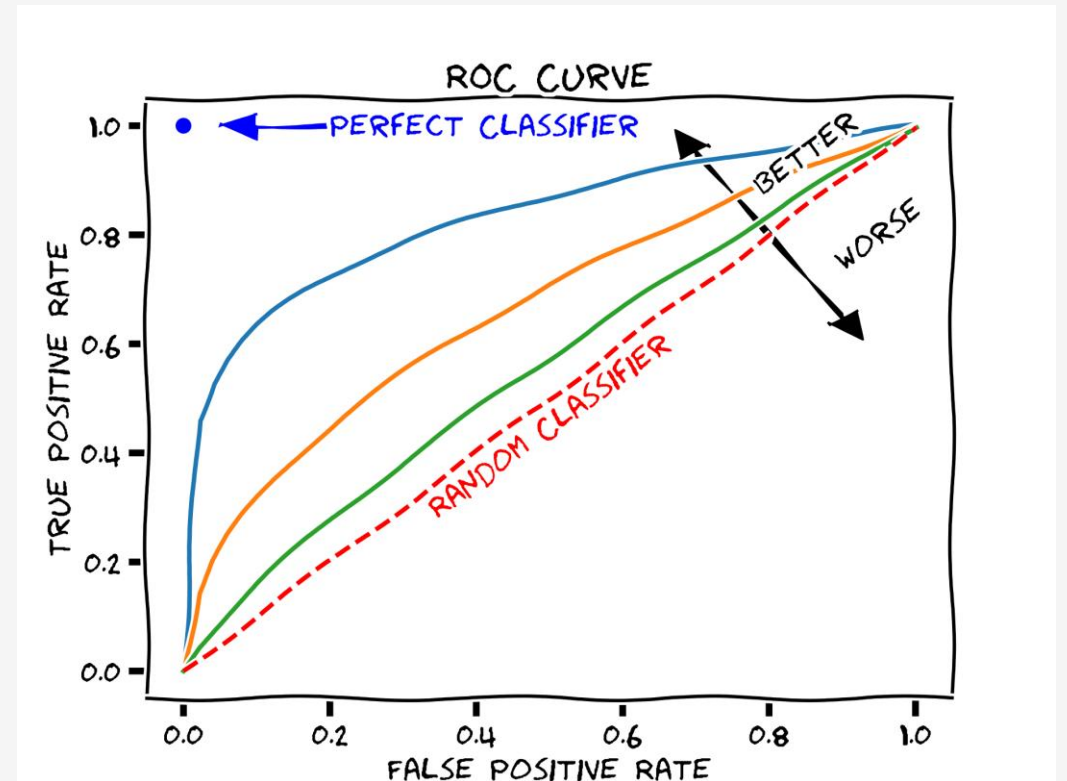
### F1-Score

$$\begin{aligned} F1 &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \\ &= 2 \times (0.91 \times 0.83) / (0.91 + 0.83) = 0.87 \end{aligned}$$

- 의미: 정밀도와 재현율의 조화평균
- 추천: 불균형 데이터의 기본 지표

## ROC-AUC

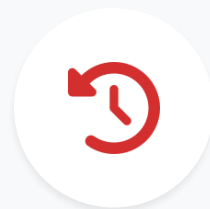
- ROC 곡선
  - 임계값 변화에 따른 TPR vs FPR 그래프
- AUC (Area Under Curve)
  - 0.5 (랜덤) ~ 1.0 (완벽)
- 추천: 확률 기반 모델 평가





### 신경망 구조

신경망은 결국 선형 변환과 활성화 함수의 반복적인 결합



### 역전파 (Backprop)

출력의 오차를 입력 방향으로 보내며 각 파라미터의 기울기를 계산하는 핵심 과정



### 활성화 함수

단순한 선형 결합을 넘어 복잡한 패턴을 학습하게 하는 비선형성(Non-linearity)의 열쇠